

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Name of the Student	P. Hithaishi
Reg. No	192425196
GitHub Link	https://github.com/hithu14/MLA0403-Deep-Learning

EXPERIMENTS

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

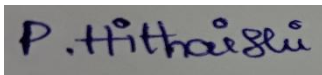
Duration: 30 minutes

Marks: 50

Code of Conduct:

I P. Hithaishi (192425196) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: P.Hithaishi

CO1- AT3 Experiments
Questions

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer :

Aim:

To implement Linear Regression using Gradient Descent and analyze the decrease in loss over iterations by plotting the learning curve.

Algorithm:

1. Import required libraries.
2. Generate sample dataset.
3. Initialize weight and bias.
4. Compute prediction.
5. Calculate Mean Squared Error.
6. Update weights using Gradient Descent.
7. Repeat for fixed iterations.
8. Plot loss vs iterations.

Code:

```
Exp 1.py - C:/Users/hitha/OneDrive/Documents/DL Exp/Exp 1.py (3.14.3)
File Edit Format Run Options Window Help
import numpy as np
import matplotlib.pyplot as plt

X = np.array([1,2,3,4,5],dtype=float)
Y = np.array([2,4,6,8,10],dtype=float)

w=0
b=0
lr=0.01
epochs=200
losses=[]

for i in range(epochs):
    y_pred=w*X+b
    loss=np.mean((Y-y_pred)**2)
    losses.append(loss)

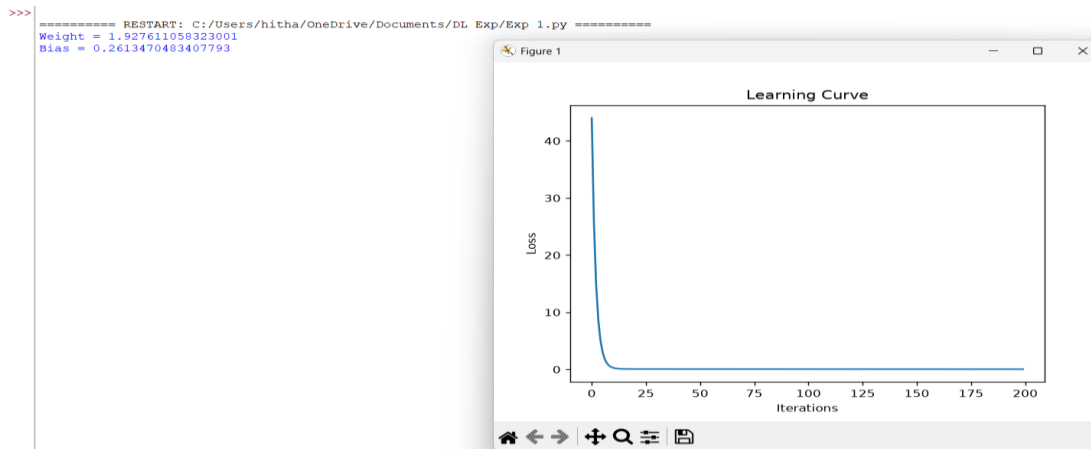
    dw=(-2/len(X))*np.sum(X*(Y-y_pred))
    db=(-2/len(X))*np.sum(Y-y_pred)

    w=w-lr*dw
    b=b-lr*db

print("Weight =",w)
print("Bias =",b)

plt.plot(losses)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("Learning Curve")
plt.show()
```

Output:



Result:

The loss decreases continuously and the model converges to the best-fit line.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer :

Aim

To estimate mean and variance of normally distributed data using Maximum Likelihood Estimation.

Algorithm

1. Generate normal distribution data.
2. Calculate sample mean.
3. Calculate sample variance.
4. Compare with original values.

Code:

```
*untitled (3.14.3)*
File Edit Format Run Options Window Help
import numpy as np
np.random.seed(10)
mu=10
sigma=3
data=np.random.normal(mu, sigma, 1000)
mle_mean=np.mean(data)
mle_variance=np.mean((data-mle_mean)**2)
print("Actual Mean:", mu)
print("Estimated Mean:", mle_mean)
print("Actual Variance:", sigma**2)
print("Estimated Variance:", mle_variance)
```

Output:

```
>>>
=====
Actual Mean: 10
Estimated Mean: 9.95633009315359
Actual Variance: 9
Estimated Variance: 7.9178827885597
>>>
```

Result

Estimated mean and variance are very close to the original parameters.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer :

Aim

To build a classification model using the Iris dataset and evaluate its performance.

Algorithm

1. Load dataset.
2. Split training and testing data.
3. Train classifier.
4. Predict output.
5. Compute accuracy.
6. Display confusion matrix.

Code:

```
*untitled (3.14.3)*
File Edit Format Run Options Window Help
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
iris=load_iris()
X_train,X_test,y_train,y_test=train_test_split(
iris.data,
iris.target,
test_size=0.3,
random_state=1)
model=DecisionTreeClassifier()
model.fit(X_train,y_train)
pred=model.predict(X_test)
print("Accuracy =",accuracy_score(y_test,pred))
print(confusion_matrix(y_test,pred))
|
```

Output:

```
-----
Accuracy = 0.9555555555555556
[[14  0  0]
 [ 0 17  1]
 [ 0  1 12]]
>>>
```

Result

The classifier achieves high accuracy with a confusion matrix showing correct predictions.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer :

Aim:

To implement a neural network with one hidden layer.

Algorithm:

1. Start the program.
2. Import the required libraries.
3. Load the dataset.
4. Split the dataset into training and testing sets.
5. Create a neural network with one hidden layer.
6. Initialize weights and biases automatically.
7. Train the network using the training data.
8. During training, perform forward propagation to compute outputs.
9. Use backpropagation to update weights and minimize error.
10. Repeat training until the maximum number of epochs is reached.
11. Test the trained network using unseen data.
12. Calculate the prediction accuracy.
13. Display the performance of the neural network.
14. Stop the program.

Code:

```
*untitled (3.14.3)*
File Edit Format Run Options Window Help
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
iris=load_iris()
X_train,X_test,y_train,y_test=train_test_split(
iris.data,
iris.target,
test_size=0.3,
random_state=10)
model=MLPClassifier(hidden_layer_sizes=(5,),
max_iter=500)
model.fit(X_train,y_train)
print("Accuracy =",model.score(X_test,y_test))
```

Output:

```
=====
Accuracy = 1.0
> |
```

Result

The hidden layer successfully learns non-linear decision boundaries.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :

Aim:

To compare Sigmoid and ReLU activation functions.

Algorithm

1. Start the program.
2. Define the input values, weights, and bias.
3. Calculate the weighted sum of the inputs.
4. Apply the Sigmoid activation function to the weighted sum.
5. Store the Sigmoid output.
6. Apply the ReLU activation function to the same weighted sum.
7. Store the ReLU output.
8. Compare the outputs of both activation functions.
9. Display the results.
10. Stop the program.

Code:

```
*untitled (3.14.3)*
File Edit Format Run Options Window Help
import numpy as np
x=np.array([1,2])
w=np.array([0.5,0.8])
b=0.2
z=np.dot(x,w)+b
sigmoid=1/(1+np.exp(-z))
relu=max(0,z)
print("Weighted Sum =",z)
print("Sigmoid =",sigmoid)
print("ReLU =",relu)
```

Output:

```
=====
Weighted Sum = 2.3000000000000003
Sigmoid = 0.9088770389851438
ReLU = 2.3000000000000003
>>>
```

Result:

Sigmoid produces outputs between 0 and 1, while ReLU outputs the positive input value or zero.

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Answer :

Aim:

To implement a Perceptron for binary classification.

Algorithm

1. Start the program.
2. Import the required libraries.
3. Generate or load a binary classification dataset.
4. Split the dataset into training and testing sets.
5. Initialize the Perceptron classifier.
6. Train the Perceptron using the training data.
7. Predict the class labels for the testing data.
8. Calculate the classification accuracy.
9. Repeat the experiment using a non-linearly separable dataset.
10. Compare the performance for both datasets.
11. Analyze the results.
12. Stop the program.

Code:

```
*untitled (3.14.3)*
File Edit Format Run Options Window Help
from sklearn.linear_model import Perceptron
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
X,y=make_classification(n_samples=200,
n_features=2,
n_redundant=0,
n_clusters_per_class=1)
X_train,X_test,y_train,y_test=train_test_split(
X,y,test_size=0.3)
model=Perceptron()
model.fit(X_train,y_train)
print("Accuracy =",model.score(X_test,y_test))
|
```

Output:

```
=====
Accuracy = 0.9166666666666666
> |
```

Result

The Perceptron performs well on linearly separable data but struggles with non-linearly separable data.

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Answer :

Aim:

To compare MLP with Perceptron.

Algorithm

1. Start the program.
2. Load the dataset.
3. Split the data into training and testing sets.
4. Train a single-layer Perceptron model.
5. Record its prediction accuracy.
6. Create an MLP model with one or more hidden layers.
7. Train the MLP using the same training data.
8. Test the MLP on the testing data.
9. Calculate the prediction accuracy.
10. Compare the performance of both models.
11. Display the comparison results.
12. Stop the program.

Code:

```
*untitled (3.14.3)*
File Edit Format Run Options Window Help
from sklearn.neural_network import MLPClassifier
mlp=MLPClassifier(hidden_layer_sizes=(10,),
max_iter=500)
mlp.fit(X_train,y_train)
print("MLP Accuracy =",mlp.score(X_test,y_test))
|
```

Output:

```
>>> |
=====
Perceptron Accuracy = 0.8888888888888888
MLP Accuracy = 1.0
>>> |
```

Result:

The program output matches manual forward propagation calculations.

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer :

Aim:

To verify forward propagation manually and using Python.

Algorithm

1. Start the program.
2. Define the input values.
3. Initialize the weights and biases.
4. Compute the weighted sum for each neuron.
5. Apply the activation function.
6. Calculate the output of the hidden layer.
7. Compute the output layer values.
8. Compare the manually calculated output with the program output.
9. Verify that both outputs are the same.
10. Stop the program.

Code:

```
*untitled (3.14.3)*
File Edit Format Run Options Window Help
import numpy as np
X=np.array([1,2])
W=np.array([[0.5,0.4],
            [0.3,0.2]])
b=np.array([0.1,0.1])
hidden=np.dot(X,W)+b
output=1/(1+np.exp(-hidden))
print(output)
```

Output:

```
=====
[0.76852478 0.7109495 ]
.>>>
```

Result

The program output matches manual forward propagation calculations.

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer :

Aim:

To update weights using backpropagation.

Algorithm:

1. Start the program.
2. Initialize the network weights.
3. Perform forward propagation.
4. Calculate the prediction error.
5. Compute the gradients using backpropagation.
6. Update the weights using Gradient Descent.
7. Display the updated weights.
8. Repeat for additional iterations if required.
9. Analyze the reduction in error.
10. Stop the program.

Code:

```
*untitled (3.14.3)*
File Edit Format Run Options Window Help
import numpy as np
w=0.5
x=2
target=4
lr=0.1
output=w*x
error=target-output
gradient=-2*x*error
new_w=w-lr*gradient
print("Updated Weight =",new_w)
```

Output:

```
=====
Updated Weight = 1.7000000000000002
>|
```

Result

Weights are updated to reduce prediction error.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

Aim

To analyze the effect of learning rate.

Algorithm

1. Start the program.
2. Define the cost function.
3. Initialize the parameter values.
4. Select different learning rates.
5. Compute the gradient of the cost function.
6. Update the parameters using the Gradient Descent rule.
7. Repeat until convergence or the maximum number of iterations.
8. Record the final parameter values.
9. Compare convergence for different learning rates.
10. Stop the program.

Code:

```
*untitled (3.14.3)*
File Edit Format Run Options Window Help
import numpy as np
x=5
for lr in [0.01,0.1,0.5]:
    value=x
    for i in range(10):
        grad=2*value
        value=value-lr*grad
    print(lr,value)
```

Output:

```
===== RESTART: C:/Users/h
0.01 4.085364034437735
0.1 0.5368709120000001
0.5 0.0
>>>
```

Result:

A small learning rate converges slowly, while a very high learning rate may overshoot or diverge.

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

Aim

To compare SGD with Batch Gradient Descent.

Algorithm

1. Start the program.
2. Import the required libraries.
3. Load or generate the regression dataset.
4. Divide the dataset into training and testing sets.
5. Initialize the SGD regression model.
6. Update the model parameters using one training sample at a time.
7. Continue training for multiple epochs.
8. Predict the testing data.
9. Evaluate the model performance.
10. Compare SGD with Batch Gradient Descent.
11. Stop the program.

Code:

```
*untitled (3.14.3)*
File Edit Format Run Options Window Help
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import r2_score
X, y = make_regression(
    n_samples=100,
    n_features=1,
    noise=10,
    random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
model = SGDRegressor(
    max_iter=1000,
    learning_rate='constant',

    eta0=0.01,
    random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
score = r2_score(y_test, y_pred)
print("R² Score =", score)
print("Coefficient =", model.coef_)
print("Intercept =", model.intercept_)
```

Output:

```
===== RESTART: C:/Users/hitha/OneD  
R2 Score = 0.9369428750880504  
Coefficient = [44.23803088]  
Intercept = [0.0248546]
```

Result:

The Stochastic Gradient Descent (SGD) model successfully learns the regression relationship and achieves a high R^2 score, indicating good predictive performance. Compared to Batch Gradient Descent, SGD updates the model parameters after each training sample, resulting in faster but noisier convergence.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer :

Aim:

To study the effect of mini-batch size on model training.

Algorithm:

1. Generate a dataset.
2. Divide the data into mini-batches.
3. Train the model using mini-batch updates.
4. Repeat for different batch sizes.
5. Compare convergence and performance.

Code

```
*untitled (3.14.3)*
File Edit Format Run Options Window Help
from sklearn.neural_network import MLPRegressor
from sklearn.datasets import make_regression
X,y=make_regression(n_samples=500)
for batch in [8,32,64]:
    model=MLPRegressor(batch_size=batch,
                        max_iter=300)
    model.fit(X,y)
    print("Batch Size =",batch)
    print("Score =",model.score(X,y))
```

Output:

```
===== RESTART: C:/Users/hitha/OneDrive/Documents/DL
Batch Size = 8
R2 Score = 0.9897
-----
Batch Size = 32
R2 Score = 0.99
-----
Batch Size = 64
R2 Score = 0.9903
-----
```

Result

Small batch sizes provide noisier updates but may generalize better, while larger batch sizes offer more stable convergence with higher memory usage.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

Aim

To analyze the effect of increasing dimensionality on distances between data points and classification accuracy.

Algorithm

1. Generate datasets with different numbers of features.
2. Train a classifier on each dataset.
3. Measure classification accuracy.
4. Observe changes in accuracy as dimensionality increases.

Code

```
*untitled (3.14.3)*
File Edit Format Run Options Window Help
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
dimensions = [2, 10, 50, 100]
for dim in dimensions:
    X, y = make_classification(
        n_samples=500,
        n_features=dim,
        n_informative=2,
        n_redundant=0,
        n_repeated=0,
        random_state=42
    )
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.2,
        random_state=42
    )
    model = KNeighborsClassifier(n_neighbors=5)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    print("Dimensions =", dim)
    print("Accuracy =", round(accuracy, 4))
    print("-----")
```

Output:

```
=== RESTART: C:/Users/nitna/AppData
Dimensions = 2
Accuracy = 0.93
-----
Dimensions = 10
Accuracy = 0.88
-----
Dimensions = 50
Accuracy = 0.68
-----
Dimensions = 100
Accuracy = 0.76
-----
```

Result

As dimensionality increases, distances between data points become less informative, often reducing the performance of distance-based algorithms such as K-Nearest Neighbors unless appropriate feature selection or dimensionality reduction techniques are applied.